# Почему ours больше не учится на NeMo

Обучение драфта EAGLE-3 с нуля на H200: срывы рецепта NeMo AutoModel, их причина и что запущено сейчас

Команда EAGLE-3, большое обучение

13 октября 2026

**Коротко**

1. **NeMo-тренер на нашей задаче нестабилен.** Пять попыток, четыре срыва обучения при lr от 2e-4 до 2e-5; ни одна не прошла дальше шага ≈930. Код авторов при lr 5e-5 прошёл больше 28 тыс. шагов без единого срыва.
2. **Причина — рост логитов внимания q·k.** Внутри срыва градиенты по `q_proj`, `k_proj` и `fc` в 14–27 раз больше, чем у здоровой модели, а выходной слой не меняется.
3. **Главное отличие от кода авторов — инициализация.** Веса драфта NeMo стартуют в 3 раза крупнее (разброс 0,02 против 0,0065), поэтому оценки внимания на старте в ≈9,5 раза больше. Большой батч, высокий lr и bf16 усиливают эффект.
4. **Решение (13.10 00:12):** оба варианта учатся кодом авторов и отличаются только входом драфта — слоями и `fc_norm`. Сравнение стало чище.
5. **Промежуточный результат:** ours впереди baseline в каждой точке валидации на 0,015–0,03 τ при равном числе диалогов. Итог — в точке 546 600 диалогов, реальной τ в SGLang.

## 1 Что сравниваем и что запущено сейчас

Два драфта EAGLE-3 для LLaMA-3.1-8B-Instruct учатся с нуля одним и тем же кодом авторов (SafeAILab/EAGLE, `traineagle3`) на одних данных в одном порядке, на одной H200. Сессии SLURM длятся не больше 8 часов; обучение сохраняется перед концом сессии и продолжается в следующей с того же диалога.

**Как читать τ-оценку**

τ — среднее число токенов, которое драфт угадывает за один проход большой модели; чем больше, тем быстрее генерация. τ-оценка на валидации считается из точности 7 шагов драфта (как у авторов): $1 + a_1 + a_1 a_2 + \ldots$. Это быстрый прокси; итоговое сравнение — реальная τ в SGLang для обоих драфтов в точке сравнения.

## 2 Что пробовали с NeMo

Изначально ours должен был учиться рецептом NeMo AutoModel для LLaMA-3.1-8B на PerfectBlend: lr 2e-4 с косинусом, 4 шага TTT, 32 упакованные строки по 2048 токенов на шаг (≈75 диалогов), плюс ускорения из задачи 4 — упаковка коротких диалогов и `torch.compile`. Данные,

| | **baseline** | **ours** |
|---|---|---|
| Слои цели (вход слоя $i$) | 2, 16, 29 — как в статье | 16, 22, 29 ($N/2$, 0,7$N$, $N-3$) |
| Перед `fc` | — | `fc_norm`: RMSNorm на каждый из 3 слоёв с обучаемым масштабом (как в EAGLE-3.1) |
| Тренер | код авторов, DeepSpeed fp16, 7 шагов TTT | то же |
| Батч, lr | 8 диалогов на шаг, lr 5e-5 | то же |
| Данные | PerfectBlend-Regenerated-Llama-3.1-8B, 1 366 504 диалога | то же, тот же порядок |
| Остановка | 0,4 эпохи: шаг 68 325 = 546 600 диалогов | то же |
| Прогресс на 13.10 11:25 | шаг 28 600, 229 тыс. диалогов | шаг 13 400, 107 тыс. диалогов |
| Последняя валидация | τ-оценка 3,49 на 224 тыс. | τ-оценка 3,14 на 96 тыс. |

**Таблица 1:** Два варианта. Отличаются только тем, какие скрытые состояния модели-цели получает драфт и как они нормируются.

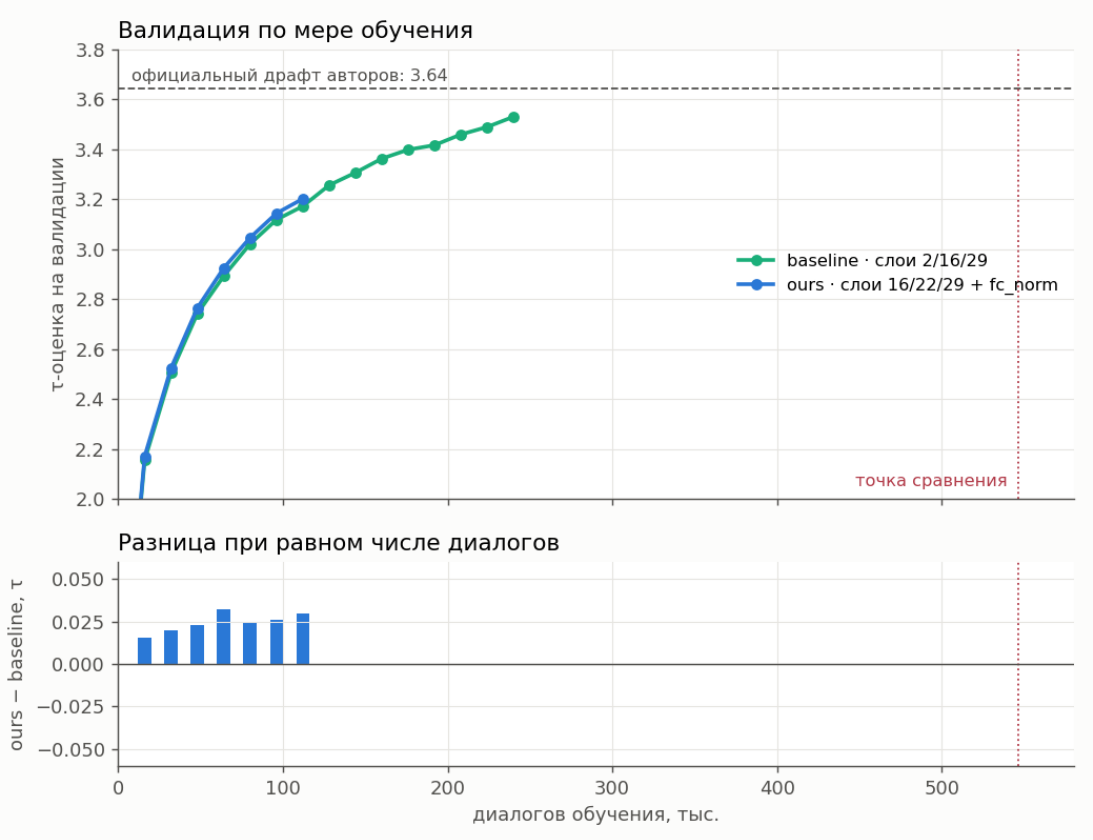



**Рис. 1:** Сверху — τ-оценка на 400 отложенных диалогах по мере обучения; пунктир — официальный драфт авторов на той же валидации (3,64), вертикальная линия — точка сравнения. Снизу — разница ours − baseline при равном числе диалогов (τ baseline интерполирована между его валидациями). ours впереди во всех семи точках, разница пока небольшая: 0,015–0,03.

порядок и словарь драфта мы подставили те же, что у baseline, а валидацию — мерой авторов, чтобы кривые были сравнимы.

| № | Режим | Что случилось | Итог |
|---|---|---|---|
| 1 | Рецепт как опубликован: веса и AdamW в bf16, lr → 2e-4 | Обновления меньше половины шага bf16 округляются в ноль: все 7 RMSNorm, включая `fc_norm`, остались ровно 1,0 после 477 шагов. На шаге 470 (lr 1,04e-4) начался скачок loss 4,30 → 5,46 | Остановлен на шаге 477 |
| 2 | fp32 мастер-веса, вычисления в bf16; lr → 2e-4 | Учился лучше варианта 1 (τ 1,88 против 1,70 на 32 тыс.), но на шаге 580 (lr 1,28e-4) loss 3,16 → 3,57 → 8,1, норма градиента 1 → $9 \cdot 10^4$ → $10^{18}$, точность → 0 | Разошёлся |
| 3 | fp32, пик lr 1e-4 + защита: пропуск шага при норме > 10, откат при норме > 100 | С шага ≈809 (lr 8,9e-5) нормы 13–16, к шагу 880 — 102 | Автооткат к 723, lr ×0,5 |
| 4 | То же, пик lr 5e-5 — как у авторов | На шаге 859, через 70 шагов после продолжения, нормы 18–70; пропущено 187 шагов из ≈250, модель застыла (loss 3,3 полчаса) | Остановлен вручную, откат к 790 |
| 5 | С шага 790, пик lr 2e-5; пропуск только inf/NaN | До шага 920 норма 0,8–1,0, на шаге 930 — 167 | Автооткат, lr 1e-5; остановлен на 922 |

**Таблица 2:** Пять попыток NeMo-тренера за 12.10 и ночь на 13.10.

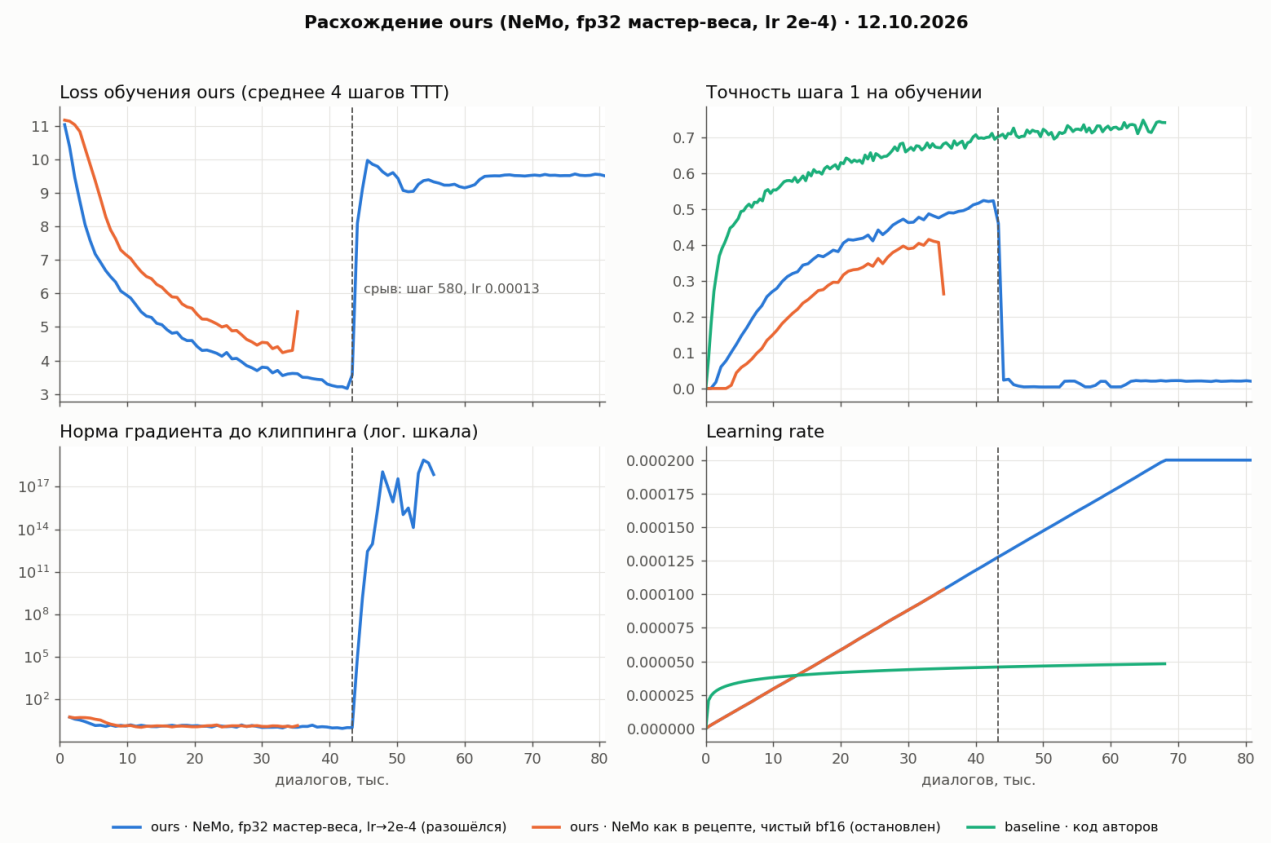



**Рис. 2:** Попытки 1 (оранжевая) и 2 (синяя) рядом с baseline (зелёная). Пунктир — срыв попытки 2 на шаге 580. До срыва fp32-вариант учится быстрее bf16 (слева сверху loss ниже, справа сверху точность выше), но в момент срыва норма градиента (слева снизу, логарифмическая шкала) взлетает на 18 порядков. Справа снизу видно, что срыв произошёл ещё во время разгона lr. Точность baseline выше на тех же диалогах: у него в 9 раз больше шагов оптимизатора на диалог.

**Закономерность:** срыв каждый раз наступал около шагов 580–930, а меньший lr лишь откладывал его. При 5e-5 он случился через 70 шагов после продолжения со здорового чекпоинта, при 2e-5 — через 140. Значит, к шагу ≈800 что-то накапливается в самой модели, а lr решает только, насколько быстро это проявится.

## 3 Почему взрывались градиенты

Чтобы найти, что рвётся, мы прогнали одни и те же 32 упакованные строки вперёд и назад на двух чекпоинтах — здоровом (шаг 790) и изнутри срыва (шаг 933) — и посчитали норму градиента по каждому весу драфта (режим `train_nemo_full.py --diagnose`).




**Рис. 3:** Норма градиента по весам драфта на одних и тех же строках. Внутри срыва больше всего выросли градиенты весов внимания `q_proj` и `k_proj` (×25–27) и питающего их `fc` (×14). Выходной слой `lm_head` почти не изменился, loss тоже (3,22 → 3,35).

Это известная нестабильность трансформеров — **рост логитов внимания**:

1. Оценки «насколько позиция A важна для позиции B» — произведения $q \cdot k$ — постепенно растут, и softmax внимания сваливается почти целиком на одну позицию.
2. В таком состоянии крошечное изменение $q$ или $k$ резко перебрасывает внимание с одной позиции на другую. Поэтому градиенты по `q_proj`, `k_proj` и `fc` становятся в десятки раз больше.
3. Сами веса при этом не растут: их среднеквадратичное значение не изменилось. Растёт их согласованность. Loss сначала почти тот же, потом очередной шаг оптимизатора делает слишком большой скачок, и модель разваливается.

**Почему не помогли клиппинг и малый lr**

**Клиппинг** (1,0) ограничивает длину вектора градиента. Срыв же начинался при норме ≈1, и клиппинг просто не включался. К тому же Adam делит градиент на свою оценку его масштаба, и шаг каждого веса остаётся порядка lr при любой длине градиента — ограничивает его только сам lr.

**Малый lr** уменьшает каждый шаг, но к шагу ≈800, пройденному на высоком lr, «заострение» внимания уже накоплено. Меньший шаг его не убирает, а только откладывает срыв.

**Пропуск шагов** с большой, но конечной нормой оказался ловушкой: пропущенный шаг не меняет веса, и модель застревает в плохом состоянии навсегда. Чекпоинты 933 и 1052 дали в диагностике совпадающие до цифры результаты. Пропускать можно только шаги с inf/NaN,

как это делает DeepSpeed.

Стандартное средство от роста логитов — **QK-norm**: RMSNorm на $q$ и $k$ внутри внимания, так что их произведение не может расти неограниченно. Но это другая архитектура драфта: её нужно учить с нуля (≈40 часов GPU) и добавить в наш класс модели для SGLang. К утру четверга это не успевает.

## 4 Чем NeMo отличается от кода авторов

**Главное измеренное отличие — инициализация весов.** Авторы создают слои драфта стандартным `nn.Linear` из PyTorch, у которого разброс весов $\approx 1/\sqrt{3 \cdot d_{\text{вход}}}$. NeMo строит драфт как модель HuggingFace и инициализирует все слои с разбросом 0,02, как большую LLM. За первые сотни шагов веса почти не сдвигаются, поэтому измерение на чекпоинтах — это и есть инициализация.

| Вес (среднеквадратичное значение) | Код авторов, шаг 998 | NeMo, шаг 723 |
|---|---|---|
| `q_proj`, `k_proj`, `v_proj` (вход 8192) | 0,0065 | 0,020 |
| `o_proj`, MLP gate/up (вход 4096) | 0,009 | 0,020 |
| `fc` (вход 12288) | 0,0053 | 0,020 |
| `lm_head` | 0,011 | 0,020 |

**Таблица 3:** Масштаб весов драфта, измеренный на чекпоинтах обоих тренеров.

Входы $q$ и $k$ нормированы (RMS ≈1), поэтому масштаб оценок внимания легко посчитать. Разброс одного элемента $q$ равен $\sigma_w\sqrt{d_{\text{вход}}}$, а логит $q \cdot k/\sqrt{d_{\text{головы}}}$ имеет разброс $\approx \sigma_w^2\, d_{\text{вход}}$:

$$\text{авторы: } 0{,}0065^2 \cdot 8192 \approx 0{,}35, \qquad \text{NeMo: } 0{,}02^2 \cdot 8192 \approx 3{,}3.$$

У авторов внимание на старте почти равномерное и мягкое, у NeMo — сразу «острое», то есть в ≈9,5 раза ближе к опасной зоне.

| Отличие | NeMo | Код авторов | Вероятная роль в срыве |
|---|---|---|---|
| Инициализация | разброс 0,02 | 0,005–0,011 | **главная** (измерено) |
| Батч на шаг | 32 строки ≈ 75 диалогов | 8 диалогов | усиливающая: чистый градиент толкает веса в одну сторону (гипотеза) |
| Пик lr и разгон | 2e-4, линейно за 5% шагов | 5e-5, логарифмически за 12 000 шагов | усиливающая: высокий lr ускорил накопление |
| Точность | bf16 (7 бит мантиссы); в рецепте даже веса и AdamW | fp16 (10 бит) с масштабированием loss, fp32 мастер-веса, пропуск шагов с переполнением | усиливающая: большие логиты в bf16 считаются грубее |
| Шаги TTT | 4 | 7 | вероятно нет |
| Нормировка loss | взвешенное среднее по шагам | взвешенная сумма | нет: Adam нечувствителен к масштабу loss |
| Упаковка диалогов | несколько в строке 2048 | один на пример | нет: маски не смешивают диалоги (проверено) |
| Клиппинг | 1,0 | 0,5 | нет: срыв начинался при норме ≈1 |

**Таблица 4:** Отличия рецепта NeMo от кода авторов.

Инициализация как главная причина — вывод по измерениям и расчёту. Прямой проверки (NeMo с инициализацией как у авторов) пока не было.

## 5 Наши модификации

Модель, loss и обучение с TTT у обоих вариантов — код авторов без изменений. Меняли только вход драфта, цикл обучения под 8-часовые сессии и настройки под одну GPU.

**В коде авторов** (`patch_eagle.py`, `train.py`):

- выбор слоёв цели из настроек (у авторов слои 2/16/29 зашиты в код) и модуль `fc_norm` перед `fc` — и в обучении, и в их коде инференса;
- цикл с чекпоинтами полного состояния (веса, оптимизатор, масштаб loss, позиция в данных): каждые 45 минут, за 12 минут до конца сессии SLURM, по сигналу и по файлу STOP; продолжение ровно с того же диалога;
- DeepSpeed: накопление 8 на одной GPU (у авторов 4 GPU × 2), расписание lr на наше число шагов вместо зашитых 800 000;
- валидация на 400 отложенных диалогах каждые 2000 шагов, экспорт весов каждые 10 000 шагов.

**В рецепте NeMo** (`train_nemo_full.py`, сейчас не используется): те же данные, порядок и словарь драфта, что у baseline; валидация мерой авторов; fp32 мастер-веса; продолжение с той же строки данных (NeMo восстанавливает только номер шага); автооткат при расхождении; режим диагностики градиентов.

**Инфраструктура:** `session.py` запускает оба варианта в каждой сессии и перезапускает упавший; метрики идут в ClearML, тревоги и часовая сводка с графиками — в Telegram-бот. По ходу исправлены неверный конец сессии (в окружении не было `SLURM_JOB_ID`) и зависание синхронизации при таймаутах ClearML (статус и графики теперь в разных процессах).

## 6 Что дальше

baseline дойдёт до точки сравнения примерно через 22 часа GPU, ours — ещё через 3–4 часа. С паузами между сессиями это среда 14.10, день или вечер. После этого — реальная τ обоих драфтов в SGLang и прогноз, сколько ещё нужно учить.

Если вернёмся к NeMo, путей два:

1. **Инициализация как у авторов** (разброс $\approx 1/\sqrt{3 \cdot d_{\text{вход}}}$ вместо 0,02). Архитектура не меняется, совместимость с SGLang сохраняется. Проверка — около часа: если на рецептном lr шаги 800–1000 проходят без роста норм $q/k$, причина доказана. Запустим, когда baseline остановится и освободит часть GPU.
2. **QK-norm** — надёжное средство, но требует обучения с нуля и поддержки в SGLang; в идеи на потом.

Логи, графики и чекпоинты всех прогонов NeMo сохранены: на сервере — `runs/ours_bf16_stopped`, `runs/ours_fp32_lr2e-4_diverged`, `runs/ours_nemo_attention_unstable`; локально — `big_training/reports/incidents`.

## Приложение: словарь

**Драфт**
Маленькая модель (один слой трансформера), которая угадывает несколько следующих токенов большой модели. Большая модель проверяет догадки за один проход; чем больше угадано, тем быстрее генерация.

**τ (acceptance length)**
Среднее число токенов, принятых за один проход большой модели.

**TTT (training-time test)**
Приём EAGLE-3: на обучении драфт несколько шагов подряд продолжает собственные предсказания, как на инференсе. У авторов 7 шагов, в рецепте NeMo — 4.

**lr (learning rate)**
Размер шага, с которым меняются веса за одно обновление. Слишком большой шаг — обучение «перескакивает» и разваливается.

**Норма градиента**
Длина вектора градиента — насколько сильно loss хочет изменить веса. В здоровом обучении здесь 1–1,6.

**Клиппинг**
Ограничение длины градиента сверху перед шагом оптимизатора.

**bf16, fp16, fp32**
Форматы чисел: 16 бит с 7 битами мантиссы, 16 бит с 10 битами, 32 бита. Мастер-веса в fp32 — веса хранятся точно, а считаются быстро в 16 битах.

**Логиты внимания**
Оценки $q \cdot k/\sqrt{d}$, из которых softmax делает веса внимания. Большие логиты — «острое» внимание почти на одну позицию.

**QK-norm**
Нормализация $q$ и $k$ внутри внимания; не даёт логитам расти неограниченно.

**fc_norm**
Наша нормализация (как в EAGLE-3.1): каждое из трёх скрытых состояний цели нормируется своим RMSNorm с обучаемым масштабом перед смешивающим слоем `fc`.